

Mashbot — SonarQube Report Interpretation

SE3002 Assignment 01, Part 3A — SonarQube Report and NFR Evaluation

Tool: SonarQube Cloud (sonarcloud.io), Automatic Analysis. **Project:** mashbot2.0

(github.com/Abdullah-SE-bit/mashbot2.0). **Analyzed commit:** 74bd4f8a, branch master (tagged baseline-v2).

Codebase size: 2.1k lines of code across 49 files. **Scope:** the complete frozen codebase was scanned, not a subset.

Top-line results (Context)

Quality area	Rating	Open issues
Security	C	1
Reliability	A	0
Maintainability	A	45

Duplicated lines: **2.1%** project-wide (of 2.5k lines analyzed for duplication), concentrated in `src/lib/actions/users.ts` (18.4%, 26 duplicated lines) and `src/lib/actions/content.ts` (11.2%, 26 duplicated lines). Coverage was not measured (no automated test suite was configured for this MVP — see the Limitation column below).

Interpretation of the top-line ratings: a Maintainability rating of A alongside 45 open issues is not a contradiction — the letter rating reflects the ratio of estimated remediation effort to the size of the codebase, not the issue count. This is an important limitation to state explicitly: a good rating does not mean zero defects, only that the estimated cost to fix what was found is small relative to project size. Likewise, 0 open Reliability issues means SonarQube's static rules found no classic "Bug"-type findings — it does not mean the application has no functional defects; several of the genuine defects confirmed through manual testing (see Part 3B) were classified by SonarQube under Maintainability rather than Reliability (see Finding 1 below), which is itself a notable interpretive point.

Five selected findings, interpreted

Selected to span the required interpretive dimensions (what was reported, where, why it matters, and what action the evidence supports), and to include one finding whose maintainability signal turned out to trace back to a genuine confirmed functional defect (Finding 1).

Finding 1 — Useless assignment hides a real login-security defect

File: `src/lib/actions/auth.ts`, line 78 | Quality area: Maintainability | Severity: Medium | SonarQube message: "Remove this useless assignment to variable 'isDeactivated'."

What SonarQube reported: SonarQube flagged that the local variable `isDeactivated` is computed (from the user's profile status) but never read afterward.

Where it occurs: Inside the `signIn` server action, immediately after the Supabase authentication call succeeds.

Why it matters: This is the clearest example in the whole report of why a "cosmetic" Maintainability finding deserves real investigation rather than a one-line fix: the discarded value is the result of the account-deactivation check required by SRS 0390/0650. Because the value is computed and then dropped, a deactivated account is never actually blocked from logging in. This was independently confirmed by manual testing (see BUG-01 / test case TC-02 in Part 3B) — the SonarQube finding and the functional defect are the same root cause, found by two different methods.

What action the evidence supports: Do not simply delete the unused variable (which would silence the warning without fixing anything). Restore the early return/sign-out that should follow the check, so a deactivated account is rejected at login

as the SRS requires.

Finding 2 — Cognitive Complexity far exceeds the allowed threshold

File: `src/lib/statusHelpers.ts`, line 4 | Quality area: Maintainability | Severity: High | SonarQube message: "Refactor this function to reduce its Cognitive Complexity from 31 to the 15 allowed."

What SonarQube reported: The function `describeContentStatus()` scores 31 on SonarQube's Cognitive Complexity metric, roughly double the default threshold of 15.

Where it occurs: A single function that builds a tooltip string by nesting conditionals over content status, user role, and content ownership.

Why it matters: High cognitive complexity correlates directly with defect risk: a function with this many nested branches is difficult to reason about, difficult to test exhaustively (each nested condition multiplies the number of paths), and easy to break silently when a new `ContentStatus` value is introduced elsewhere in the codebase.

What action the evidence supports: Refactor into either a lookup table keyed by (status, role, ownership) or a sequence of early-return guard clauses, one condition at a time. Add a unit test per branch first so the refactor can be verified to preserve existing behaviour.

Finding 3 — Duplicated / unreachable branch in status rendering

File: `src/components/campaigns/status-badge.tsx`, line 25 | Quality area: Maintainability | Severity: Medium | SonarQube message: "This branch's code block is the same as the block for the branch on line 7."

What SonarQube reported: The final `else` branch of an `if/else-if` chain duplicates the code already handled by the first branch (the "draft" case).

Where it occurs: `StatusBadge()`, the component that renders a content item's status pill.

Why it matters: Given the TypeScript `ContentStatus` union type, this branch can never actually execute for a well-typed value — it is dead code left over from converting a clean lookup table into an `if/else` chain. Dead branches like this are a maintainability risk specifically because they look like defensive coding but actually mask the fact that an unhandled status would silently render as "Draft" rather than surfacing an error.

What action the evidence supports: Remove the duplicate branch, or better, revert to a typed lookup object/map so the TypeScript compiler enforces that every status is handled exactly once.

Finding 4 — Repeated literal in the database migration

File: `supabase/migrations/0001_init.sql`, line 14 | Quality area: Maintainability | Severity: High | SonarQube message: "Define a constant instead of duplicating this literal 5 times." (a second instance, 3 times, was also flagged at line 45)

What SonarQube reported: The same string literal appears five separate times within `CHECK` constraints/policy definitions in the schema migration.

Where it occurs: The `profiles/campaign_content` table definitions in the initial migration.

Why it matters: This is schema-level duplication with an application-level consequence: the same set of allowed values (e.g. role or status names) exists independently in this SQL file and again as TypeScript union types in `src/lib/types.ts`. A future rename in one place that is not mirrored in the other would produce a silent mismatch between what the database allows and what the application believes is valid — exactly the kind of gap that surfaces as a hard-to-reproduce production bug.

What action the evidence supports: Introduce a Postgres `ENUM` type (or a single referenced constant list) for each repeated value set, and use it everywhere instead of repeating the literal.

Finding 5 — Duplication concentrated in the authorization-check files

Files: src/lib/actions/users.ts (18.4%, 26 duplicated lines), src/lib/actions/content.ts (11.2%, 26 duplicated lines) | Quality area: Maintainability / Context | Metric: Duplicated Lines (%)

What SonarQube reported: SonarQube's duplication detector reports these two files as the most internally repetitive in the project, well above the 2.1% project-wide average.

Where it occurs: Both files implement the server actions for the Contributor/Approver/Publisher workflow (SRS 0140–0230) and user-account management (SRS 0240/0330/0380) — i.e. exactly the files that enforce who is allowed to do what.

Why it matters: Duplication is a maintainability metric in general, but its location here has security relevance: each of these files repeats a similar “load profile → check role/status” pattern independently for every action. If an authorization rule needs to change, a duplicated copy is easy to miss, producing an inconsistency between two actions that are supposed to enforce the same rule.

What action the evidence supports: Extract the repeated “load current profile and check role/status” logic into a single shared helper (as was already done for admin checks via `requireAdmin()`) and have both files call it, so a future authorization change only needs to be made once.

Evidence note: the underlying screenshots (SonarQube Cloud project summary and the individual issue rows quoted above, from mashbot2.0, commit 74bd4f8a) are retained separately as the literal screenshot evidence required by the assignment brief; this document is the written interpretation of that evidence, not a replacement for it.